

Cup Manipulator: Modes, Kinematics & Pulley Routing

`script_cup_manipulator_pydrake.py` — `test_drive_pulley.py`

Isaac-sim robotics notes

March 2026

1 Script Overview: `script_cup_manipulator_pydrake.py`

The script drives a 2-DOF revolute cup manipulator (joints `link1_base` q_1 , `link2.link1` q_2) with an optional 3-D pendulum ball attached to link 2. It is invoked via `--mode`:

Mode	Description
<code>scene-viz</code>	Static Meshcat display, no physics
<code>simulation</code>	Full PhysX dynamics, PD torque control
<code>joint-motion</code>	Sinusoidal joint sweep, excites ball swing
<code>run-all-jts</code>	Interactive terminal: type angles, robot moves
<code>min-jerk-joint</code>	Smooth point-to-point with minimum-jerk profile

2 Forward Kinematics (2-DOF Planar)

With link lengths $L_1 = 342.47$ mm and $L_2 = 190.00$ mm (both lying in the horizontal XY plane at $Z = \text{const}$):

$$\mathbf{p}_{ee} = \begin{pmatrix} x_{ee} \\ y_{ee} \end{pmatrix} = \begin{pmatrix} L_1 \cos q_1 + L_2 \cos(q_1 + q_2) \\ L_1 \sin q_1 + L_2 \sin(q_1 + q_2) \end{pmatrix} \quad (1)$$

Drake computes this implicitly; the code reads the EE position via:

```
cup_body = plant.GetBodyByName("link2", model_instance)
X_WC = plant.CalcRelativeTransform(context,
                                   plant.world_frame(),
                                   cup_body.body_frame())
p_ee = X_WC.translation() # [x, y, z] world frame
```

3 Minimum-Jerk Joint-Space Profile (min-jerk-joint mode)

Given a move of duration T and normalised time $s = t/T \in [0, 1]$, the minimum-jerk velocity profile is:

$$h(s) = 10s^3 - 15s^4 + 6s^5 \quad (2)$$

$$\dot{h}(s) = \frac{30s^2 - 60s^3 + 30s^4}{T}, \quad \ddot{h}(s) = \frac{60s - 180s^2 + 120s^3}{T^2} \quad (3)$$

$h(0) = 0$, $h(1) = 1$; first and second derivatives vanish at both endpoints, giving smooth (C^2) motion with zero jerk at start and end.

```

def min_jerk_profile(t: float, T: float):
    s = np.clip(t / T, 0.0, 1.0)
    h = 10*s**3 - 15*s**4 + 6*s**5
    hdot = (30*s**2 - 60*s**3 + 30*s**4) / T
    hddot = (60*s - 180*s**2 + 120*s**3) / T**2
    return h, hdot, hddot

# Apply to joint space
q_cmd = q_start + h * (q_target - q_start)

```

4 Script Overview: test_drive_pulley.py

Visualises the tendon/cable routing of the cable-driven manipulator in Meshcat. The cable transmits torque from the drive pulley on body q_1 (pulley_htd_5m_60t) through two idler bearings (623ZZ) to the large driven pulley on body q_2 (link2_tendon).

Cable route (ordered):

Start anchor $\rightarrow$ Drive pulley (HTD 5M 60T, $r_p \approx 47.75$ mm) $\rightarrow$ Idler-L / Idler-R (623ZZ) $\rightarrow$ Big driven pulley (q_2 body) $\rightarrow$ End anchor

5 URDF Visual-Origin Transform

Every pulley centroid is computed from its OBJ mesh centre $\mathbf{p}_{\text{local}}$ and the URDF <visual><origin> tag:

$$\mathbf{p}_{\text{body}} = R_z(\psi) R_y(\theta) R_x(\phi) \mathbf{p}_{\text{local}} + \mathbf{r}_{\text{xyz}} \quad (4)$$

where (ϕ, θ, ψ) are roll-pitch-yaw from the URDF rpy attribute.

```

# PulleyBase._compute_centroid()
roll, pitch, yaw = self.vis_rpy
Rx = np.array([[1,0,0],[0,np.cos(roll),-np.sin(roll)],
               [0,np.sin(roll), np.cos(roll)]])
Ry = ... # standard Ry(pitch)
Rz = ... # standard Rz(yaw)
R = Rz @ Ry @ Rx

centroid_body = R @ centroid_local + np.array(self.vis_xyz)

```

6 External Tangent Between Two Pulleys

To route the cable between two circular pulleys with centres $\mathbf{c}_1, \mathbf{c}_2$ (world frame) and radii r_1, r_2 , the external tangent contact points T_1, T_2 satisfy:

$$D = \|\mathbf{c}_2 - \mathbf{c}_1\|, \quad \hat{\mathbf{d}} = \frac{\mathbf{c}_2 - \mathbf{c}_1}{D}, \quad \hat{\mathbf{n}}_{\perp} = (-\hat{d}_y, \hat{d}_x) \quad (5)$$

$$\cos \alpha = \frac{r_1 - r_2}{D}, \quad \sin \alpha = \sqrt{1 - \cos^2 \alpha} \quad (6)$$

$$\mathbf{n} = \cos \alpha \hat{\mathbf{d}} + \epsilon \sin \alpha \hat{\mathbf{n}}_{\perp}, \quad \epsilon \in \{+1, -1\} \quad (7)$$

$$T_1 = \mathbf{c}_1 + r_1 \mathbf{n}, \quad T_2 = \mathbf{c}_2 + r_2 \mathbf{n} \quad (8)$$

ϵ selects the upper/lower branch. For an *internal* tangent (cable crosses between pulleys) replace $\cos \alpha = (r_1 + r_2)/D$ and negate r_2 in T_2 .

```
# PulleyBase.compute_tangent() (works in XY; Z preserved)
d = p2 - p1 # 2-D displacement
D = np.linalg.norm(d)
d_hat = d / D
perp = np.array([-d_hat[1], d_hat[0]]) # CCW normal

cos_a = (r1 - r2) / D # external tangent
sin_a = np.sqrt(max(0.0, 1.0 - cos_a**2))
n = cos_a * d_hat + branch * sin_a * perp # branch = +/-1

T1 = np.array([p1[0] + r1*n[0], p1[1] + r1*n[1], c1[2]])
T2 = np.array([p2[0] + r2*n[0], p2[1] + r2*n[1], c2[2]])
```

After computing tangents in the world frame, Drake FK maps them back into the respective body frames for cable drawing:

```
# _Xw returns (R, t) for a body in the world frame
R1, t1 = _Xw(cfg1.body_name)
c1_w = R1 @ cfg1.centroid + t1 # centroid in world

T1_w, T2_w = PulleyBase.compute_tangent(
    c1_w, cfg1.radius, c2_w, cfg2.radius, branch)

# Back to body frame (for FK consistency in draw_cables)
T1_body = R1.T @ (T1_w - t1)
T2_body = R2.T @ (T2_w - t2)
```

7 Pulley Pitch Radius (HTD 5M 60T)

Both the drive and driven pulleys share the same HTD 5M 60T belt spec:

$$r_p = \frac{N \cdot p}{2\pi} = \frac{60 \times 5 \text{ mm}}{2\pi} \approx 47.75 \text{ mm} \quad (9)$$

```
belt_teeth = 60
belt_pitch_m = 0.005 # 5 mm pitch (HTD-5M)
pitch_radius = belt_teeth * belt_pitch_m / (2 * np.pi) # ~0.04775 m
```